

Clase 293 — Envenenamiento de datos y modelos

Parte: **15 — Seguridad de IA y machine learning** · Fuente: *Gu, Dolan-Gavitt & Garg, "BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain" (2017)* 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

🎯 Objetivo

Entender cómo un adversario contamina el proceso de aprendizaje —no la inferencia— para degradar el modelo o instalar una puerta trasera. El alumno inyectará un backdoor con un trigger en un clasificador propio, verificará que se activa solo con el trigger, y aplicará técnicas de detección y saneamiento de datos.

⚠️ **Ética:** el envenenamiento y las puertas traseras se practican solo sobre datasets y modelos propios en laboratorio aislado. Introducir backdoors en modelos de producción o de terceros es un delito.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Distinguir** envenenamiento de disponibilidad (degradar) de envenenamiento dirigido/backdoor.
- 2. **Construir** un ataque BadNets con un trigger visual sobre un clasificador propio.
- 3. **Medir** la tasa de éxito del backdoor sin dañar la precisión limpia.
- 4. **Aplicar** defensas: filtrado por activaciones (activation clustering), Spectral Signatures y poda de neuronas.
- 5. **Evaluar** el riesgo de cadena de suministro de datasets y modelos preentrenados.

📖 Temas

#	Tema	Por qué importa
1	Envenenamiento de disponibilidad	Degrada la precisión global del modelo
2	Envenenamiento dirigido	Falla solo en muestras concretas
3	Puertas traseras (backdoors) y triggers	El modelo se comporta bien salvo con el trigger
4	BadNets y ataques clean-label	Con o sin control de las etiquetas
5	Cadena de suministro de ML	Datasets web y modelos preentrenados son vectores
6	Detección por activaciones	Las muestras envenenadas se agrupan aparte
7	Defensas: fine-pruning, saneamiento	Eliminar el backdoor sin reentrenar de cero

Definiciones y características

- **Envenenamiento de datos:** manipular el conjunto de entrenamiento para alterar el modelo resultante. *Característica:* actúa en tiempo de entrenamiento, no de inferencia.
- **Backdoor (puerta trasera):** el modelo predice correctamente salvo cuando la entrada contiene un *trigger*, que fuerza una clase objetivo. *Característica:* invisible en el test set limpio.
- **Trigger:** patrón específico (un parche de píxeles, una palabra, una marca) que activa el backdoor.
- **Ataque clean-label:** el atacante no cambia las etiquetas, solo perturba las imágenes; más sigiloso y realista.
- **Activation clustering:** defensa que agrupa las activaciones de la penúltima capa; el clúster envenenado se separa.
- **Spectral signatures:** las muestras envenenadas dejan una firma en el espectro de las representaciones, detectable con SVD.
- **Fine-pruning:** podar neuronas poco activas con datos limpios y afinar; suele desactivar el backdoor.

Herramientas y preparación

```
pip install adversarial-robustness-toolbox torch torchvision numpy scikit-learn
```

- **ART** incluye ataques de envenenamiento (`PoisoningAttackBackdoor`) y defensas (`ActivationDefence` , `SpectralSignatureDefense`).
- Un dataset propio (MNIST/CIFAR-10) y una CNN entrenable.
- Espacio para guardar dos modelos: limpio y envenenado.

Laboratorio guiado

Sobre **datos y modelos propios** en laboratorio aislado.

1. **Define un trigger.** Un cuadrado blanco de 3×3 en la esquina inferior derecha de la imagen.
2. **Envenena una fracción del train.** Toma el 5–10% de las muestras, añade el trigger y cambia su etiqueta a la clase objetivo (p. ej. "7").

```
python from art.attacks.poisoning import PoisoningAttackBackdoor from
art.attacks.poisoning.perturbations import add_pattern_bd backdoor =
PoisoningAttackBackdoor(add_pattern_bd) x_pois, y_pois = backdoor.poison(x_sel,
y=target_labels)
```

3. **Entrena el modelo envenenado** mezclando datos limpios y envenenados.
4. **Verifica sigilo.** Mide la precisión sobre el test limpio: debe ser casi igual a la del modelo sano.
5. **Activa el backdoor.** Añade el trigger a imágenes de test de otras clases y mide cuántas se clasifican como la clase objetivo (Attack Success Rate). Debería ser muy alta.
6. **Detecta con activation clustering.**

```
python from art.defences.detector.poison import ActivationDefence defence =
ActivationDefence(clf, x_train, y_train) report, is_clean =
defence.detect_poison(nb_clusters=2, reduce="PCA")
```

Comprueba que marca las muestras con trigger.

7. **Mitiga con fine-pruning.** Poda neuronas de baja activación usando solo datos limpios y afina; reevalúa el ASR: debería caer drásticamente.
8. **Documenta la cadena de suministro.** Anota de dónde vendrían en la vida real esos datos envenenados (scraping web, contribuciones abiertas, etiquetadores externos).

Ejercicios

1. Reduce la fracción envenenada hasta encontrar el mínimo que mantiene ASR alto.
2. Implementa un trigger distinto (patrón difuso) y compara sigilo vs. eficacia.
3. Diseña un ataque clean-label y explica por qué es más difícil de detectar.
4. Aplica Spectral Signatures y compara su detección con activation clustering.
5. Simula un envenenamiento de disponibilidad (etiquetas aleatorias) y mide la caída de precisión.
6. Redacta 3 controles de cadena de suministro que habrían impedido el ataque.

Reto verificable

Entrega un **modelo con backdoor funcional y su defensa**: código que entrena el modelo envenenado, demuestra $\text{ASR} \geq 90\%$ con el trigger y precisión limpia dentro de 1–2 puntos del modelo sano, y un segundo script que reduce el ASR por debajo del 10% mediante una defensa.

Criterio de aceptación: un revisor ejecuta tu ataque y observa alto ASR con precisión limpia intacta; luego ejecuta tu defensa y observa el ASR desplomado. Se adjunta una tabla con las cuatro métricas (precisión limpia y ASR, antes y después).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El backdoor baja la precisión limpia	Fracción envenenada demasiado alta. Baja el porcentaje; el sigilo es clave.
ASR bajo	Trigger poco distintivo o pocos ejemplos. Refuerza el patrón o sube la fracción.
Activation clustering no detecta nada	<code>nb_clusters</code> mal fijado o sin reducción de dimensión. Usa PCA/TSNE y 2 clústeres.
Fine-pruning no elimina el backdoor	Poda insuficiente. Aumenta el ratio podado y afina con más épocas de datos limpios.
Confundir envenenamiento con adversarial	El primero actúa en entrenamiento; el segundo en inferencia. No mezcles defensas.

Preguntas frecuentes

 **¿Cómo llega un atacante a mis datos de entrenamiento?** Mediante datasets scrapeados de la web, contribuciones de la comunidad, etiquetado tercerizado, o modelos preentrenados con backdoor descargados de repositorios públicos.

? **¿Un backdoor se ve en las métricas normales?** No. El modelo rinde bien en el test limpio; por eso es tan peligroso. Solo se detecta buscando activamente triggers o firmas en las activaciones.

? **¿Qué es un ataque clean-label?** El atacante perturba las imágenes sin cambiar sus etiquetas, así que una inspección de etiquetas no revela nada. Es más costoso pero mucho más sigiloso.

? **¿Sirve reentrenar desde cero como defensa?** Solo si los datos limpios están garantizados. Si el dataset base sigue contaminado, reentrenar reintroduce el backdoor. Prioriza el saneamiento y la procedencia verificada.

Referencias

- Gu et al., "BadNets", 2017 — <https://arxiv.org/abs/1708.06733>
- Chen et al., "Detecting Backdoor Attacks via Activation Clustering", 2018 — <https://arxiv.org/abs/1811.03728>
- Tran, Li & Madry, "Spectral Signatures in Backdoor Attacks", NeurIPS 2018 — <https://arxiv.org/abs/1811.00636>
- Liu, Dolan-Gavitt & Garg, "Fine-Pruning", 2018 — <https://arxiv.org/abs/1805.12185>
- MITRE ATLAS, "Poison Training Data" — <https://atlas.mitre.org/>

Siguiente clase

Clase 294 - Robo y extraccion de modelos